Autonomous LLM-Driven RTL Generation for Deep Neural Networks via Multi-Agent Orchestration

DANIEL BOTEZATU*, University of Twente, The Netherlands

We present nn2rtl (neural network to register-transfer level, or RTL), an autonomous multi-agent large language model (LLM) pipeline that translates trained, quantised PyTorch convolutional networks [17] into independently verified, on-chip-only synthesisable Verilog for field-programmable gate arrays (FPGAs), integrated through the Model Context Protocol [2] under a hard constraint forbidding external dynamic random-access memory (DRAM). Two cooperating AI systems produced all the RTL and ran the entire tool flow; the author directed the work and built the deterministic pipeline and verification environment the agents run and self-correct inside, writing no RTL and running no commands. A first system generated and verified per-layer modules; a second built the shared compute engine, debugged the design end to end, and ran the synthesis and place-and-route campaign, largely autonomously. We generated and verified all three networks byte-exact against their quantised references: the ResNet-50 convolutional backbone, MobileNetV2, and ResNet-8. On an Alveo U250 we routed MobileNetV2 at 110.90 MHz and closed the ResNet-50 backbone's timing at 83.33 MHz. In a cross-flow ResNet-8 comparison on a ZCU104 against FINN [27] and hls4ml [21], nn2rtl fitted and routed at 142.86 MHz with 87.19% top-1 accuracy, whereas the most accurate flow, hls4ml at 89.10%, did not fit. We characterised the observed effectiveness of LLM assistance across the design stages (RQ1), compared performance, area, and estimated power against deterministic flows (RQ2), and contributed a verification methodology with a bug taxonomy (RQ3).

Additional Key Words and Phrases: large language models, RTL generation, Verilog, neural network accelerators, multi-agent systems, FPGA, quantisation

1 INTRODUCTION

State-of-the-art neural networks are typically developed as pre-trained models in high-level frameworks such as PyTorch [17]. Deploying a trained convolutional neural network (CNN) on a field-programmable gate array (FPGA), a reconfigurable chip whose logic is wired into the shape of the target computation, lets the compute and memory architecture be specialised to the target network for low-latency, energy-efficient inference [4, 18]. Realising that efficiency, however, requires translating the network into register-transfer level (RTL) hardware, a description of the circuit in terms of registers and the logic between them, that can then be synthesised (compiled from RTL into a gate-level netlist), functionally verified against the original model, and evaluated for power, performance, and area. This translation is a critical but time-consuming engineering step.

Large language models (LLMs) have recently shown promise at generating code, including hardware description languages, but RTL remains a difficult target: it must satisfy strict spatial and temporal constraints, and small errors break compilation, simulation, or synthesis. Prior work on LLM-assisted RTL has largely focused on small, isolated modules such as counters and finite state machines, leaving integration and debugging to the human engineer. Compile-and-repair loops improve pass rates over single-shot generation [22], retrieval-augmented repair resolves most syntax errors [26], and multi-agent decompositions raise quality on benchmark-scale problems [10, 30, 32]. None of these targets the neural network accelerator domain at the scale of a full network, nor carries a design through to a placed-and-routed FPGA result.

nn2rtl (neural network to RTL) is an autonomous multi-agent LLM system that takes a trained, quantised PyTorch CNN [17] and produces verified, synthesisable Verilog using only on-chip memory for FPGAs. The agents coordinate with synthesis and verification tools through the Model Context Protocol (MCP) [2], a standard interface between an LLM and external tools. We deployed and routed two large designs on an AMD Alveo U250 accelerator card, the ResNet-50 [9] convolutional backbone and the complete MobileNetV2 [20], and ran a cross-flow comparison of a smaller ResNet-8 on a ZCU104 board against the two established deterministic flows, FINN [27] and hls4ml [6, 21]. All three networks were independently verified byte-exact against their quantised reference, and all three were placed and routed (the back-end mapping onto physical FPGA sites).

Two cooperating AI systems produced all RTL and executed the synthesis and verification flow, while the author set goals, approved milestones, resolved design forks, and built the deterministic pipeline and verification environment the agents run and self-correct inside. System 1 generates and verifies per-layer modules; System 2 integrates them and runs synthesis and place-and-route (Figure 1).

Our contributions are: (1) an end-to-end multi-agent pipeline that autonomously generates and independently verifies functional FPGA hardware for full CNNs; (2) two large designs taken from trained PyTorch checkpoints to placed-and-routed implementations on an Alveo U250, the ResNet-50 convolutional backbone and the complete MobileNetV2, plus a routed complete ResNet-8 on a ZCU104; (3) a cross-flow performance and area comparison against FINN and hls4ml, with estimated power against FINN; and (4) a verification methodology and failure taxonomy for trusting LLM-generated hardware. A hard constraint shaped the entire design: all weights and activations remain in on-chip memory, with no external DRAM. We impose this deliberately: external DRAM would move the memory bottleneck off chip and add platform-dependent bandwidth and energy costs, whereas the on-chip-only constraint gives a fixed budget shared by the compared designs. This on-chip-only requirement made FPGA memory the binding resource and drove key architectural decisions, most notably a single shared,

*Supervisor: Amirreza Yousefzadeh (CAES). Coordinator: Ghayoor Gillani (CAES).

TScIT 45, July 3, 2026, Enschede, The Netherlands

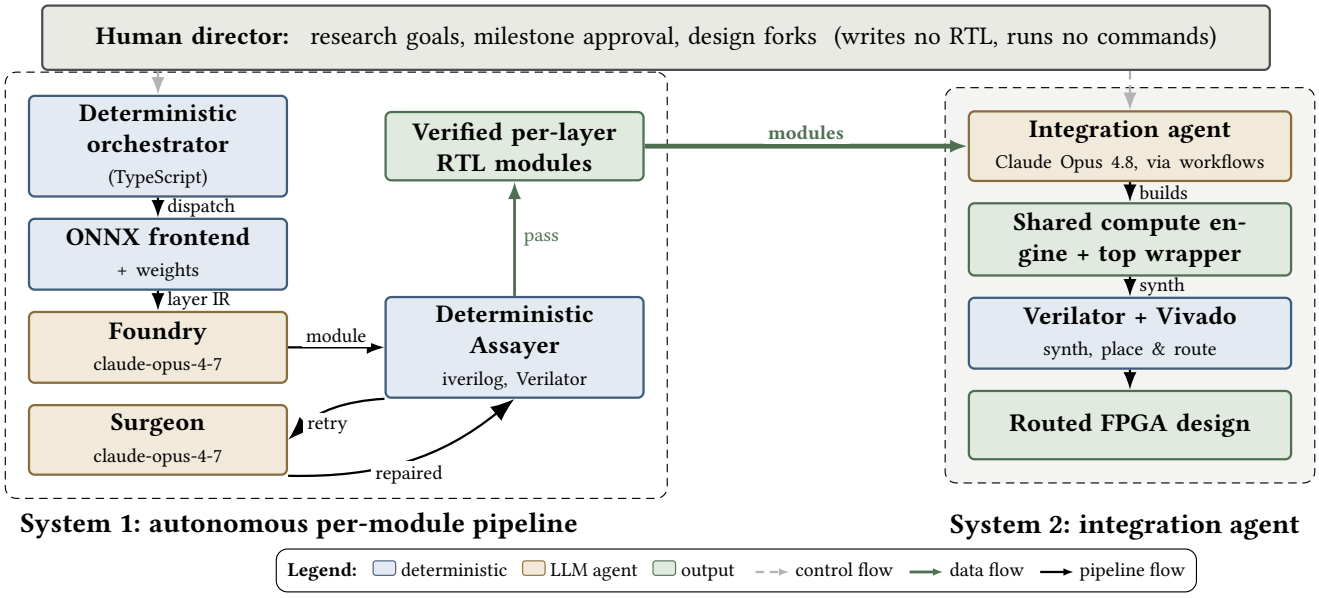

Fig. 1. The two cooperating AI systems: System 1 generates and verifies per-layer RTL modules; System 2 assembles, debugs, and routes the full design.

time-multiplexed compute engine in place of separate datapaths for each heavy convolution.

This work is guided by three research questions:

(1) Across the stages of model-to-RTL generation, including architecture mapping, RTL synthesis from the network specification, and closed-loop repair, where does LLM assistance contribute, and which stages still require deterministic or hand-written components?
(2) What is the resulting implementation quality, in terms of power, performance, and area (PPA), compared with deterministic FPGA-targeted workflows such as FINN and hls4ml?
(3) What verification is needed to trust LLM-generated hardware, independent of how it was produced: what classes of integration and correctness bugs arise, and what methodology detects them, given that individually correct modules can still fail once composed into a full design?

The remainder of the paper is organised as follows. Section 2 surveys related work, Section 3 describes the two-system methodology and the verification and quantisation flows, Section 4 details the experimental setup, Section 5 reports results against the three research questions, and Section 6 concludes.

2 BACKGROUND AND RELATED WORK

The intersection of large language models (LLMs) and hardware design has advanced quickly, but important gaps remain in scale, autonomy, and domain specialisation. The work in this paper sits at the meeting point of three threads: using LLMs to write register-transfer-level (RTL) hardware code, applying them specifically to AI accelerators, and the deterministic FPGA flows that have long served as the reference point for such accelerators. We review each in turn and then state the gap this work fills.

2.1 LLM-Assisted RTL Generation

Tomlinson et al. [25] used ChatGPT to generate the complete Verilog for a programmable spiking-neuron-array ASIC, verified it with hand-written testbenches, and taped it out through Tiny Tapeout, establishing early feasibility at small scale; the verification and tapeout were human-driven rather than part of a closed autonomous loop. Two developments then put generation on firmer ground. First, functional benchmarks and fine-tuned open models made model-level RTL generation measurable: VerilogEval [13] and RTLLM [15] are standard functional benchmarks, and RTLCoder [14] shows that a fine-tuned open model can rival much larger proprietary ones. Second, compile-and-repair loops feed the compiler or simulator error back to the generator: AutoChip [22] drives successive rounds of correction from compiler feedback, and RTLFixer [26] adds retrieval-augmented repair, resolving roughly 98.5% of the compilation errors in its debugging dataset. Together these results established that an outer loop of automated checking turns an unreliable one-shot generator into a usable RTL producer, but they target mostly isolated, benchmark-scale modules.

2.2 Multi-Agent RTL Generation

A natural next step was to split the task across cooperating agents rather than asking one model to do everything. VerilogCoder [10] combines a graph planner with repair guided by syntax trees and waveforms. RTLSquad [30] and MAGE [32] decompose generation into specialised roles for exploration, implementation, and verification, reporting quality gains on benchmark-scale Verilog problems

and echoing a broader move towards automating the design of agentic systems themselves [11]. These systems are primarily evaluated on self-contained, benchmark-scale RTL tasks; nn2rtl instead studies model-derived, composed neural network hardware through network-level verification and FPGA implementation.

2.3 LLMs for AI-Accelerator Design

A separate line of work pointed LLMs directly at accelerator generation. GPT4AIGChip [8] drove an LLM pipeline that produced high-level-synthesis (HLS) accelerator templates through demonstration-augmented prompting. TPU-Gen [29] combines fine-tuned models and retrieval-augmented generation with automated validation and a refinement loop to generate parameterised systolic-array tensor-processing-unit implementations. Neither, however, verifies the composed hardware layer-by-layer against the quantised execution of a source CNN, nor studies full network integration under an all-weights-on-chip constraint, the regime nn2rtl targets.

2.4 Autonomous End-to-End Hardware Generation

A recent end-to-end demonstration is the original Design Conductor [24], a non-peer-reviewed technical report that used an LLM-driven workflow to take a complete RISC-V CPU from specification through to layout. This is an important proof that automation can span the whole flow, but CPUs and neural network accelerators expose different pressures. A CPU is control-heavy: its difficulty lies in instruction decoding, hazards, and branching state machines. A deep neural network (DNN) accelerator is data-intensive: it performs very regular arithmetic over large tensors, and its dominant cost is moving weights and activations on and off chip and through the compute array, the dataflow concern characterised for accelerators by Eyeriss [5]. A strategy tuned for control logic does not transfer directly to this regular-compute, movement-bound regime. The same group has since extended the harness to a transformer key-value-cache inference accelerator [23], the closest concurrent work. It is a floating-point design verified by the agent's own testbenches against a software reference, with residual numerical deviation accepted rather than exact equality required; it manages its key-value cache through external DRAM; and it reports no comparison against deterministic flows. This contrasts with the byte-exact integer verification, all-weights-on-chip constraint, and head-to-head evaluation we pursue for convolutional networks here.

2.5 Deterministic FPGA Flows Used as Baselines

The accelerators we compare against are produced by mature, deterministic toolchains that emit synthesisable hardware without any LLM involvement, which makes them fair benchmarks for whether LLM-generated RTL can approach conventional quality. FINN [4, 27] compiles quantisation-aware-trained PyTorch [17] models into per-layer streaming dataflow accelerators, with each layer folded to a chosen degree of parallelism; the FINN ResNet-50 build on the Alveo U250 [18] is the established large-network demonstration of this style. hls4ml [6, 21], which originated in particle physics, takes an HLS route and generates streaming (io_stream) dataflow at higher arithmetic precision. fpgaConvNet [28] is a further deterministic generator in this family. For the same-network ResNet-8 comparison these flows are the obvious reference, and prior hls4ml and custom high-level-synthesis ResNet-8 entries do exist [16, 31]. They are not directly usable as drop-in baselines, however: the published hls4ml ResNet-8 entry alters the residual skip connection [31], which changes the network; the closest custom HLS ResNet-8 design [16] uses a different toolchain; and there is no published FINN ResNet-8 baseline at all. For those reasons the FINN and nn2rtl ResNet-8 numbers reported here are our own builds of the unmodified network.

2.6 The Gap This Work Fills

None of these threads, on its own, covers the whole problem. What is missing is a pipeline that combines these strengths for full convolutional neural networks: a closed-loop, multi-agent system whose generated modules are checked deterministically for numerical correctness and exact timing against reference outputs, that emits RTL for the full network kept entirely on chip, and that carries the result all the way through synthesis and place-and-route to a routed, timing-closed implementation. This work fills that gap with two cooperating AI systems (Section 3.1). Prior work has shown LLM-driven accelerator-template generation, multi-agent RTL generation on benchmark tasks, and autonomous end-to-end CPU and accelerator implementation [8, 23, 24, 29, 30, 32]. We are not aware of prior work that demonstrates the complete combination studied here: LLM-agent-generated full network CNN RTL, deterministic checking against a quantised reference, an all-weights-on-chip constraint, and FPGA place-and-route.

3 METHODOLOGY AND SYSTEM DESIGN

3.1 Two Cooperating AI Systems

nn2rtl is built around a deliberate division of labour between two cooperating AI systems with different autonomy boundaries, and that division provides the structural basis for answering RQ1. The first is an autonomous *per-module pipeline* that converts a trained, quantised PyTorch CNN [17] into individually verified Verilog modules. The second is a *coding and integration agent*, a separate Claude Opus 4.8 instance, that assembles those modules into an integrated network-level accelerator, debugs it end to end, and drives the synthesis and place-and-route campaign. System 2 uses Opus 4.8 simply because it became available during the integration work as the newer state-of-the-art release; measuring its advantage over the Opus 4.7 used by System 1's generation agents was out of scope, and any 4.7-to-4.8 difference would be too small to isolate from the per-module variation in task difficulty. As stated in Section 1, the author acted as research director; every line of hardware and every tool invocation came from one of the two AI systems. The author's positive contribution is the deterministic infrastructure that makes this autonomy possible: the orchestrator that runs the loop and, at its core, the static C++ oracle and two byte-exact gates of Section 3.3 against which the agents correct their own output. A high-level directive such as "make it work" has nothing to close against without this fixed feedback loop; the agents wrote and integrated all of the hardware, but the infrastructure that let them do so reliably is the human's. We describe the second system as *largely* rather than *fully* autonomous precisely because these human decision points remain.

Figure 1 shows the overall architecture and the feedback edges that close each loop.

3.2 The Autonomous Per-Module Pipeline (System 1)

System 1 is organised as three structural layers: a Claude Code plugin that defines the agent roles and skills, an Agent SDK orchestrator built on the Claude Agent SDK that forms the deterministic control plane, and a local MCP server [2] that exposes the design tools to the agents. The orchestrator is ordinary deterministic TypeScript, not a language model: it maintains the pipeline state, selects the next action, dispatches the agents, enforces retry limits, and runs Vivado synthesis on every module that passes verification. Because the coordination is deterministic, non-determinism enters only at the explicitly invoked LLM stages; everything else (state transitions, retry limits, verification, and synthesis dispatch) is reproducible.

System 1's language model agents are the Foundry and Surgeon, supported by the Failure-Classifier and Retrospector, each pinned to a full model identifier to avoid tier-alias drift (Table B.1 in Appendix B.2); architecture mapping runs as a deterministic ONNX frontend.

Verification is performed by a deterministic component rather than an agent. The deterministic Assayer receives the generated module and its layer IR (the per-layer specification of shape, weights, and quantisation produced during architecture mapping), runs Icarus Verilog for a syntax check followed by Verilator with the static C++ testbench, and returns a schema-validated verification result carrying the numerical and timing evidence.

Failures return structured compiler or simulation evidence for bounded repair. The Failure-Classifier distinguishes ordinary code defects from architectural failures, and passing modules proceed directly to Vivado synthesis; Figure D.1 in Appendix D gives the complete state machine.

3.3 Deterministic Verification and Pattern Reuse

Functional correctness is judged by a single static, author-specified C++ testbench that is never produced by a language model (Figure D.2). In contrast to work that uses LLMs to generate testbenches [19], we deliberately keep the oracle outside the generative loop: if the same agent wrote both the design and its test, a shared misunderstanding could make a wrong module appear correct, a correlated-oracle failure we term the two-bug problem. The testbench reads a description of the module together with binary golden vectors (the reference input stimuli and the expected output activations the module must reproduce) and checks three things: every output value, the exact pipeline latency, and the valid/ready handshake timing. Crucially, the reference is the activation stream of the *quantised* model, so the checker evaluates the RTL against the deployed quantised arithmetic rather than against the original floating-point model. At the per-module level the numerical gate permits a maximum error of three integer units, though well-implemented modules typically remain within one and many are strictly bit-identical, and a module passes only when its first output also appears on exactly the contracted cycle. The assembled network end-to-end harnesses are stricter: they require strict byte-exactness, with zero mismatched bytes. Weight values are never shown to the language models: the extraction step writes them to disk and the generated RTL loads them through a standard memory-initialisation read, which keeps numerical weight data outside the LLM context and makes generation reproducible.

The pipeline also keeps a failure-memory and pattern-reuse cache: it retains classified failures and successful implementation patterns so that later related modules can reuse earlier evidence. The promotion, archival, and retrieval policy is given in Appendix B.1; the integration-bug taxonomy is reported in Section 5.3.

3.4 Integration and the Shared Compute Engine (System 2)

The second system turns a set of verified modules into integrated device-level designs that fit within a single FPGA. Implementing every convolution as its own spatial datapath is correct but far too large for the target FPGA. The integration agent therefore built a shared, time-multiplexed compute engine for the heavy convolutions while retaining cheaper operators as spatial modules; the quantified area benefit is reported in Section 5.2, and the engine's internal sub-blocks are listed in Appendix B.4. The same integration agent wired and patched the top-level wrapper, performed all end-to-end byte-exact debugging, and ran the entire synthesis and place-and-route campaign largely unattended under standing directives, including adversarial multi-agent cross-checks.

3.5 Quantisation and the On-Chip-Only Memory Constraint

The pipeline's default number format is per-tensor symmetric INT8 post-training quantisation (PTQ), applied to the PyTorch weights before generation begins, with batch-normalisation folded into the preceding convolution so that no standalone normalisation layer appears in hardware [12]. Fitting the full ResNet-50 [9] backbone on chip, however, required a lower-precision weight format. The W4A8 GPTQ [7] path (the WxAy notation denotes x-bit weights and y-bit activations) introduced per-output-channel scaling and nibble-packed weights while retaining 32-bit accumulation. The final deployable ResNet-50 configuration combined INT3 and INT4 convolutional weights to satisfy the on-chip BRAM limit. Per-output-channel scaling is necessary at this precision. MobileNetV2 [20] deploys at INT8 with per-output-channel scaling on its depthwise layers.

The memory policy is the constraint that shapes the entire design. We adopt this constraint deliberately: all weights and activations remain in on-chip memory and the design never accesses external DRAM, which keeps FPGA capacity a fixed, comparable budget rather than shifting the memory bottleneck off chip. This makes on-chip capacity the binding resource and rules out the device's abundant UltraRAM for weights: only block RAM supports bitstream content-initialisation on this target [1] (confirmed empirically as bug B40, Appendix C), so weights live in block RAM and LUT-RAM, block RAM binds the fit, and nibble packing plus selective lower precision become the weight-fit mechanisms.

Table 1. On-chip resource budgets of the two target FPGAs (device totals from Vivado).

Resource	Alveo U250	ZCU104
LUT	1,728,000	230,400
FF	3,456,000	460,800
DSP	12,288	1,728
BRAM36	2,688	312
URAM	1,280	96

4 EXPERIMENTAL SETUP

We evaluate our two-system AI pipeline on three convolutional networks across two FPGA boards.

4.1 Target Devices

We target two Xilinx UltraScale+ FPGAs. We use the first device, the Alveo U250, for ResNet-50 and the ImageNet-scale experiments. We use the second device, the smaller embedded ZCU104, for ResNet-8 and its cross-flow comparison, so that all three flows (ours, FINN, and hls4ml) compete for the same fixed budget. Table 1 lists the resource totals for both parts.

The quantisation choices that keep the weight footprint within this budget, and why block RAM (not UltraRAM) is the binding resource, are detailed in Section 3.5.

4.2 Networks and Roles

The three networks play distinct roles. *ResNet-50* [9] is our primary subject: the full backbone is generated, integrated, and placed-and-routed on the U250. The U250 ResNet-50 design emits the pre-pool $7 \times 7 \times 2048$ feature map; global average pooling and the fully connected classifier are executed in software to obtain the reported top-1, and the assembled network byte-exact gate is over the 100,352-byte feature-map output. *MobileNetV2* [20] provides an architectural contrast on the same U250, because its depthwise-separable convolutions stress the pipeline differently from ResNet-50's standard convolutions. *ResNet-8* [3] is the cross-flow baseline on the ZCU104: it is the MLCommons MLPerf Tiny pretrained model, small enough that our flow, FINN, and hls4ml can all be measured against one another on a single embedded board.

ResNet-8 needed one model transformation before our flow could reproduce it: its two stride-2 3x3 convolutions use asymmetric "same" padding, which our generator does not express directly, so we rewrote each as an equivalent zero-embedded 4x4 kernel with symmetric padding. This changes the hardware mapping but not the outputs, which we verified bit-exact against the reference on all 10,000 test images.

4.3 Datasets and Quantisation

The ImageNet-scale networks are evaluated on the same 1500-image ImageNet validation subset, disjoint from the 256-image calibration set: for ResNet-50 we report GPTQ accuracy on this subset, and the MobileNetV2 top-1 is measured on the same subset. Because this top-1 is measured over a 1500-image subset, its 95% confidence interval is roughly two points, and about three points for a difference between two figures, so accuracy gaps below this on the subset are indicative rather than significant. ResNet-8 is evaluated on the full CIFAR-10 test set of 10,000 images.

Each flow quantises in its own native way, which we preserve rather than force into a common scheme, since each flow's quality is partly a property of its quantiser. Our flow's default is per-tensor INT8 post-training quantisation with *no retraining*; the deployed networks use per-output-channel scales, quantised directly from their trained weights. We treat this no-retraining property as a feature: it removes the labelled-data and training-time cost that the comparison flows incur, and on ResNet-8 it still matches the floating-point reference's top-1. FINN quantises ResNet-8 with W4A4 quantisation-aware training (QAT) in Brevitas, and hls4ml uses W8A8 QAT in QKeras; both therefore retrain, whereas our flow does not.

4.4 Toolchain and Verification

Our flow uses Icarus Verilog for a lint pass, Verilator driven by a static C++ testbench for functional and cycle-accurate timing verification, and Vivado v2025.2 for synthesis and place-and-route. The agents reach these tools through an MCP server [2] that exposes operations to lint and simulate RTL, run synthesis and place-and-route, extract layer weights, and write generated Verilog to disk. For the cross-flow comparison, FINN v0.10.1 runs on the Vivado 2024.2 flow and hls4ml 1.3.0 runs with Vitis HLS 2024.2.

We note one fairness caveat: the Vivado version differs across the flows (2025.2 for our flow versus 2024.2 for FINN and hls4ml). Backend tool versions affect timing and utilisation results, so the cross-flow PPA numbers should be read as indicative of each flow's design choices rather than as a controlled single-variable comparison.

4.5 Metrics and Measurement Classes

For routes that meet timing we report $F_{\max}$ as the guaranteed clock, $F_{\max} = 1000/T$ for a met constraint period T in nanoseconds; ResNet-50, ResNet-8 and FINN met their constraints. For a route that does not meet its target we instead report the achievable $F_{\max} = 1000/(T - \mathrm{WNS})$, where WNS is the worst negative slack reported by Vivado: MobileNetV2's 110.90 MHz is this achievable value for a route that did not close timing at its 7 ns target. Frame throughput for our latency-bound designs follows as $\mathrm{fps} = F_{\max} \times 10^6/(\text{cycles per frame})$, where the cycle count is measured per inference; FINN, being a fully pipelined dataflow accelerator, instead sustains $\mathrm{fps} = F_{\max} \times 10^6/(\text{initiation interval})$, where the initiation interval is far smaller than the full latency.

For honesty across the three flows, these throughput figures rest on different bases and must not be read as like-for-like: nn2rtl's is cycle-accurate from a routed design, FINN's is the analytical estimate above from a routed implementation, and hls4ml has only a C-synthesis latency estimate with no route. None is an on-board silicon measurement.

Correctness is evaluated using the two gates of Section 3.3. We report total on-chip power from Vivado's post-route power estimate. Because we use vectorless estimation (no recorded switching activity file), these figures are a relative comparison rather than absolute silicon draw; confidence is low for the nn2rtl designs and medium for FINN.

4.6 Artifact availability

The pipeline, all generated RTL, the per-network golden vectors, and the post-route timing, utilisation, and power reports are available at https://github.com/bote05/RTL_LLM_CLAUDE, with the per-agent model identifiers in Table B.1. The nn2rtl figures behind Tables 2, 3, B.3, and B.4 come from in-repository reports, with the objective-directed refinement reports underlying Table B.4 included under `output/reports/`; the FINN and hls4ml figures come from their separate build hosts (FINN on Vivado 2024.2, hls4ml on Vitis HLS 2024.2), which are external to the artifact package. The networks and datasets are the public ResNet-50, MobileNetV2, ResNet-8, ImageNet, and CIFAR-10 releases under their original licences.

5 RESULTS

5.1 RQ1: Where LLM Assistance Contributes

RQ1 is best answered as a gradient rather than a single verdict: autonomy is highest for the bounded, well-specified per-module tasks and falls away for the open-ended, system-level work of integration.

Architecture mapping is the one stage that proved not to need an LLM. Its operations (graph tracing of the quantised model, batch-normalisation folding, and weight emission) are deterministic, and in the deployed pipeline they run as a deterministic ONNX frontend and a `read_weights` tool rather than a model; the designs built on the resulting layer description pass the byte-exact gate, confirming the extraction is correct without an LLM.

RTL generation, performed by the Foundry, is where LLM assistance contributes most, and its main advantage is generality. One Foundry, with no per-architecture changes and no retraining, generated the per-layer RTL for three networks spanning two architectural families (two ResNet scales and a depthwise-separable MobileNetV2), all of which pass the assembled network byte-exact gate, directly from post-training quantisation and without the per-network quantisation-aware retraining the deterministic flows require. Generation was reliable as well as general: across the ResNet-50 backbone the pipeline averaged 1.09 attempts per state entry.

Closed-loop repair, performed by the Surgeon, is valuable but situational. About 91% of state entries completed after a single Foundry attempt with no Surgeon intervention; MobileNetV2 drove more repair (1.30 attempts per entry), reflecting harder layer types such as its depthwise and ReLU6 operators. Repair was effective where invoked: ResNet-50 produced zero fail-aborts, so every module that escalated to the Surgeon was resolved.

The pipeline also supports objective-directed refinement of verified RTL: an improve operation supplies a verified module and its Vivado synthesis report to an LLM under one of seven predefined PPA objectives, reverifies each candidate with Verilator, and accepts it only when a deterministic objective-specific threshold is met. Representative per-module runs raised DSP inference and cut LUT and flip-flop use by large margins with correctness preserved (Table B.4); these are isolated demonstrations, not claimed to improve the final routed networks.

Orchestration and per-module verification are deterministic by design: the TypeScript orchestrator and the static C++ checker were never delegated to a model, keeping control flow, retry limits, and synthesis dispatch reproducible and the checker outside the generative loop to avoid the two-bug problem (Section 3.3).

Whole-network integration is the human-directed end of the gradient: System 2 (Section 3.4) performed all coding, debugging, and tool invocation, while research goals, milestones, and design forks stayed with the author.

Across all System 1 agents, the recorded LLM cost was roughly one to two US dollars per passing module (Table B.2), excluding the separate System 2 integration agent.

RQ1 therefore shows a gradient. Autonomy tracks how bounded a task is: per-module generation is nearly autonomous, while integration is human-directed. Because we did not evaluate deterministic replacements or role-removal ablations for the generative stages, these results characterise the observed effectiveness of those stages rather than the causal marginal benefit of the LLM at each.

5.2 RQ2: Implementation Quality versus FINN and hls4ml

The only same-network head-to-head is ResNet-8 on the ZCU104; the ImageNet-scale ResNet-50 and MobileNetV2 results (Alveo U250) are standalone. The comparison fixes the network and device but is a native-flow design-point comparison.

On the ZCU104, the three flows resolve the small-device wall in three different ways (Table 2). hls4ml [6, 21] reaches the highest precision and accuracy (89.10%) but does not fit: its on-chip block-RAM memory is dominated by streaming dataflow buffers (the frame-deep buffering the residual skip connections require) rather than by weights, and the weight and activation memories alone would fit comfortably. This is structural to the streaming dataflow architecture combined with a residual topology, and it is consistent with the literature, in that published hls4ml ResNet-8 entries either drop the skip connections or alter them [31], or fit only a smaller device after buffer-depth optimisation; the ResNet-8 benchmark itself is the MLPerf Tiny reference model [3]. nn2rtl sits in the middle: at INT8 it serialises the network onto one shared compute engine, which fits and is bound by digital signal processing (DSP) blocks (Table 2). FINN [4, 27] uses the lowest precision (W4A4) and substantially fewer resources. Its throughput (FINN's own analytical estimate) spans from a baseline below nn2rtl's cycle-accurate figure to a maximum-fold point that, only in that highest configuration, is the fastest of the three (Table 2).

At ImageNet scale the two designs are standalone results (Table 3). MobileNetV2 is fully routed as a complete network; its 7 ns route did not close timing, so we report the achievable signoff $F_{\max}$ rather than a guaranteed clock (Table 3). The ResNet-50 convolutional backbone fits the device, with block RAM the binding resource, and the design routes and meets timing at its target clock (Table 3). Published FINN ResNet-50 results on the U250 [18] (binary W1A2: 2,703 fps at 195 MHz, 67.27% top-1, 1,027 kLUT; packed ternary W2A2: 69.85% top-1, synthesised within U250 resource limits but failed placement) are illustrative rather than head-to-head, since FINN uses low-precision QAT against nn2rtl's mixed INT3/INT4 PTQ; hls4ml has no comparable published ResNet-50-scale dataflow.

The central area finding links area to performance. Built as individual spatial datapaths, the fourteen heaviest convolutions alone would consume 1,803,388 LUTs; the shared time-multiplexed engine

Table 2. ResNet-8 comparison on the ZCU104 using each flow's native quantisation. Power estimates are vectorless.

Metric	nn2rtl (INT8 PTQ)	FINN base (W4A4)	FINN max-fold (W4A4)	hls4ml (W8A8 QAT)
Top-1 (CIFAR-10)	87.19%	86.68%	86.68%	89.10%
Fmax	142.86 MHz	100 MHz	300.03 MHz	137.32 MHz (est.)
Throughput	about 9,670 fps	about 1,017 fps	about 32,555 fps	did not fit
Cycles/frame	14,774 (latency)	about 98,304 (II)	9,216 (II)	175,714 (C-synth)
LUT	154,188 (66.92%)	25,760 (11%)	63,739 (28%)	200,938 (87%)
DSP	1,717 (99.36%)	74 (4%)	569 (33%)	488 (28%)
BRAM	199 tiles (63.78%)	about 40 (13%)	64.5 (21%)	1,216 BRAM18† (about 194%)
FF	64,728 (14.05%)	30,824 (7%)	62,499 (14%)	100,239 (22%)
Power	7.3 W	n/a	9.1 W	n/a
Fit / route	routed, timing met	routed	routed	C-synth only, no route

FINN and hls4ml figures come from build-host reports; nn2rtl figures come from in-repository post-route reports (Section 4). †hls4ml reports block RAM in BRAM18 (half-width) units; 1,216 BRAM18 equal 608 BRAM36, and the percentage is computed on BRAM36, as for the other rows.

Table 3. Place-and-route summary for the two Alveo U250 designs (ResNet-50 backbone and complete MobileNetV2), measured at signoff.

Metric	ResNet-50 (INT3/INT4)	MobileNetV2 (INT8 per-channel)
Top-1 accuracy	77.07%	71.27%
Routed Fmax	83.33 MHz	110.90 MHz
Throughput	14.71 fps	93.61 fps
Cycles/frame	5,664,715	1,184,731
LUT	1,196,343 (69.23%)	322,628 (18.67%)
BRAM tile	2,656 (98.81%)	1,812.5 (67.43%)
DSP	6,983 (56.83%)	3,345 (27.22%)
Power (vectorless)	16.0 W	11.1 W

that replaces them, and that in the deployed design serves all seventeen heavy-layer dispatches, uses 107,268 LUTs, both measured by Vivado post-synthesis on the U250, a reduction of roughly 17 times. This is an explicit area-for-latency trade: time-multiplexing collapses the area but serialises the computation, which is precisely why the full backbone fits the device yet runs at low throughput, and it is the mechanism behind the ResNet-50 frame rate reported above.

On the accuracy and precision axis, nn2rtl's default is per-tensor symmetric INT8 post-training quantisation with batch normalisation folded; the deployed ResNet-8 and MobileNetV2 instead use per-output-channel scaling (ResNet-8 on 7 of its 9 convolutions), which is what reaches accuracy parity. ResNet-8's INT8 top-1 equals that of its floating-point reference (both 87.19%), with no retraining (Table 2). The deployed ResNet-50 uses a mixed-precision configuration (18 INT3 and 35 INT4 convolutional layers, per output channel) and reaches 77.07% as deployed; an all-INT4 configuration measured higher on the subset (79.47%, against an 80.07% floating-point baseline) but did not fit, and the mixed precision was forced by the on-chip block-RAM budget rather than chosen for accuracy. MobileNetV2 uses per-output-channel scaling on its depthwise layers (Table 3).

On the ZCU104, nn2rtl's ResNet-8 draws about 7.3 W against FINN's 9.1 W at maximum folding, so FINN reaches roughly 3.37 times the throughput for modestly higher power, 9.1 W versus 7.3 W; this is a folding and dataflow advantage (a fully pipelined design with a small initiation interval, against our latency-bound single-frame engine) rather than a clock-speed one.

Normalising throughput by power and by area yields the efficiency picture in Table B.3 of Appendix B.3, derived purely from the locked routed-Fmax, vectorless-power, and post-route-LUT figures with no additional synthesis. The one like-for-like comparison is on the ZCU104, where nn2rtl and FINN run the same ResNet-8: there FINN's W4A4 dataflow has 2.70 times higher estimated throughput per vectorless watt and 8.14 times higher throughput per thousand LUTs than nn2rtl's time-multiplexed INT8 engine. The two U250 entries are different networks, so their throughput-per-watt figures are not comparable: they reflect each workload's cost, not accelerator efficiency. We avoid cross-device energy comparisons because the platforms have substantially different static-power baselines.

RQ2 therefore has a differentiated answer. At small scale nn2rtl demonstrates feasibility and accuracy comparable to FINN: it fits where the highest-precision flow does not, but it remains materially

less PPA-efficient than FINN's highly folded dataflow implementation. At full scale it delivers a routed depthwise-separable network and a routed residual backbone under a hard on-chip-only constraint the deterministic flows were not run against. The limits are plain: the head-to-head is on one device, and the FINN throughput is derived from its routed clock and an analytical initiation interval.

5.3 RQ3: Verification for Trustworthy Generated Hardware

Trusting RTL that an LLM produced requires verification that is both independent of the generator and layered. The per-module checker is a static C++ testbench run by System 1's per-module pipeline against golden vectors generated deterministically from the quantised reference. The verification principle is independent of the RTL generator, whereas the taxonomy records the bugs encountered in this particular AI-generated system.

We use two functional-verification gates, supplemented by synthesis and implementation checks. The two gates of Section 3.3 apply: a per-module numerical-and-latency gate and a stricter byte-exact assembled network gate. Among the fresh System-1 verification records, most modules are strictly bit-identical and the remaining recorded modules satisfy the three-unit numerical tolerance; the complete evidence state is documented in Appendix A. All three networks then pass the stricter assembled network gate over their tested vectors: the routed ResNet-50 backbone with 0 of 100,352 bytes mismatching, and MobileNetV2 and ResNet-8 each with zero mismatches across their golden vectors. Because the RTL matched its quantised reference byte-exactly on the tested vectors, we report that reference's accuracy, ResNet-8's 87.19% on the full 10,000-image CIFAR-10 test set, as the deployed model's accuracy; this is not a separate on-board hardware measurement over the whole test set.

Two observations explain why per-module agreement alone does not establish trust. First, component byte-exactness does not imply end-to-end byte-exactness: a module's isolation golden is built on the same integration assumptions that may be wrong, so a locally correct module can be globally wrong. Second, summary statistics conceal bugs: a 99.9960% byte-exact layer still hid a dropped multiply-accumulate error affecting 2 of 50,176 outputs (B47); further cases are catalogued in Appendices A and C. The byte-exact end-to-end comparison is therefore the primary functional oracle in simulation.

Table A.1 in Appendix A groups the 77 bugs into twelve classes, each with the diagnostic evidence that localises it (every bug is also listed individually, with its symptom and fix, in Table C.1 of Appendix C), including the full B21 investigation and the memory-read-latency contract bug (B34) that was byte-exact in isolation but wrong once integrated. This corpus is distinct from the Surgeon per-module generation-failure classes (Appendix B.1), which catalogue faults in producing a single module rather than faults that emerge during composition.

In answer to RQ3, trusting generated RTL requires verification independent of the generator and layered across a per-module and an assembled network gate, because individually correct modules can still fail once composed. This establishes functional correctness over the tested vectors, not exhaustively or on silicon; the taxonomy is drawn from this design's bug history rather than claimed universal to all generated or hand-written hardware.

6 CONCLUSION, LIMITATIONS, AND FUTURE WORK

Two cooperating AI systems produced verified, on-chip FPGA implementations of three CNNs: the ResNet-50 convolutional backbone and complete MobileNetV2 on the Alveo U250, and complete ResNet-8 on the ZCU104, with the author acting as research director. All three passed strict assembled network byte-exact verification against their quantised references. We read this as evidence that current LLM agents can write functional, independently verified RTL and integrate a whole accelerator, not just isolated snippets.

Bounded per-module generation was the most reliable and readily verified use of the LLM, while system-level integration remained largely autonomous but human-directed. The hardware demonstrates feasibility rather than PPA superiority: the shared engine let the design fit by trading area for latency.

We are deliberate about limitations. The ResNet-8 head-to-head is on a single device; the power figures are vectorless and relative only; and "functional correctness" here means byte-exact agreement with the quantised reference over the tested vectors, not exhaustive or on-silicon validation. Future work includes improving accelerator throughput, replacing vectorless power estimates with vectored on-silicon measurements, and extending the pipeline to additional networks.

AI STATEMENT

During the preparation of this work the author used Claude (Anthropic) to assist in enhancing the writing of this paper. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the work. This is distinct from the two AI systems studied here, whose roles are described in the body.

REFERENCES

[1] AMD (Xilinx) 2025. *UltraScale Architecture Memory Resources User Guide* (v1.14 ed.). AMD (Xilinx). https://docs.amd.com/r/en-US/ug573-ultrascale-memory-resources

[2] Anthropic. 2025. Model Context Protocol Specification. https://modelcontextprotocol.io/specification/2025-11-25. Revision 2025-11-25.

[3] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, Urmish Thakker, Antonio Torrini, Pete Warden, Jay Cordaro, Giuseppe Di Guglielmo, Javier Duarte, Stephen Gibellini, Videet Parekh, Honson Tran, Nhan Tran, Niu Wenxu, and Xu Xuesong. 2021. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*.

[4] Michaela Blott, Thomas B. Preusser, Nicholas J. Fraser, Giulio Gambardella, Kenneth O'Brien, Yaman Umuroglu, Miriam Leeser, and Kees A. Vissers. 2018. FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks. *ACM Transactions on Reconfigurable Technology and Systems* 11, 3, Article 16 (2018), 16:1–16:23 pages. https://doi.org/10.1145/3242897

[5] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138. https://doi.org/10.1109/JSSC.2016.2616357

[6] Javier Duarte, Song Han, Philip Harris, Sergo Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, and Zhenbin Wu. 2018. Fast Inference of Deep Neural Networks in FPGAs for Particle Physics. *Journal of Instrumentation* 13, 07, Article P07027 (2018). https://doi.org/10.1088/1748-0221/13/07/P07027

[7] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In

The Eleventh International Conference on Learning Representations (ICLR). https://openreview.net/forum?id=tcbBPnfwxS Published as OPTQ.

[8] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. GPT4AIGChip: Towards Next-Generation AI Accelerator Design Automation via Large Language Models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323953

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. https://doi.org/10.1109/CVPR.2016.90

[10] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-Based Planning and Abstract Syntax Tree (AST)-Based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 300–307. https://doi.org/10.1609/aaai.v39i1.32007

[11] Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated Design of Agentic Systems. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=t9U3LW7JVX

[12] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713. https://doi.org/10.1109/CVPR.2018.00286

[13] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–8. https://doi.org/10.1109/ICCAD57390.2023.10323812

[14] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. https://doi.org/10.1109/TCAD.2024.3483089

[15] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727. https://doi.org/10.1109/ASP-DAC58780.2024.10473904

[16] Filippo Minnella, Teodoro Urso, Mihai T. Lazarescu, and Luciano Lavagno. 2023. Design and Optimization of Residual Neural Network Accelerators for Low-Power FPGAs Using High-Level Synthesis. arXiv:2309.15631 [cs.AR] https://arxiv.org/abs/2309.15631

[17] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 8024–8035.

[18] Lucian Petrica, Tobias Alonso, Mairin Kroes, Nicholas J. Fraser, Sorin Cotofana, and Michaela Blott. 2020. Memory-Efficient Dataflow Inference for Deep CNNs on FPGA. In *2020 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 48–55. https://doi.org/10.1109/ICFPT51103.2020.00016

[19] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*. Association for Computing Machinery, Article 18, 10 pages. https://doi.org/10.1145/3670474.3685956

[20] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

[21] Jan-Frederik Schulte, Benjamin Ramhorst, Chang Sun, Jovan Mitrevski, Nicolò Ghielmetti, Enrico Lupi, Dimitrios Danopoulos, Vladimir Loncar, Javier Duarte, David Burnette, Lauri Laatu, Stylianos Tzelepis, Konstantinos Axiotis, Quentin Berthet, Haoyan Wang, Paul White, Suleyman Demirsoy, Marco Colombo, Thea Aarrestad, Sioni Summers, Maurizio Pierini, Giuseppe Di Guglielmo, Jennifer Ngadiuba, Javier Campos, Ben Hawks, Abhijith Gandrakota, Farah Fahim, Nhan Tran, George Constantinides, Zhiqiang Que, Wayne Luk, Alexander Tapper, Duc Hoang, Noah Paladino, Philip Harris, Bo-Cheng Lai, Manuel Valentin, Ryan Forelli, Seda Ogrenci, Lino Gerlach, Rian Flynn, Mia Liu, Daniel Diaz, Elham Khoda, Melissa Quinnan, Russell Solares, Santosh Parajuli, Mark Neubauer, Christian Herwig, Ho Fung Tsoi, Dylan Rankin, Shih-Chieh Hsu, and Scott Hauck. 2026. hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware. *ACM Transactions on Reconfigurable Technology and Systems* 19 (2026). https://doi.org/10.1145/3801979

[22] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887 [cs.AR] https://arxiv.org/abs/2311.04887 Revised 2024.

[23] The Verkor Team, Ravi Krishna, Suresh Krishna, and David Chin. 2026. Design Conductor 2.0: An Agent Builds a TurboQuant Inference Accelerator in 80 Hours. arXiv:2605.05170 [cs.AR] https://arxiv.org/abs/2605.05170 arXiv preprint; non-peer-reviewed technical report.

[24] The Verkor Team, Ravi Krishna, Suresh Krishna, and David Chin. 2026. Design Conductor: An Agent Autonomously Builds a 1.5 GHz Linux-Capable RISC-V CPU. arXiv:2603.08716 [cs.AR] https://arxiv.org/abs/2603.08716 arXiv preprint; non-peer-reviewed technical report.

[25] Michael Tomlinson, Joe Li, and Andreas G. Andreou. 2024. Designing Silicon Brains Using LLM: Leveraging ChatGPT for Automated Description of a Spiking Neuron Array. In *2024 Argentine Conference on Electronics (CAE)*. IEEE, 154–159. https://doi.org/10.1109/CAE59785.2024.10487167

[26] Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. Association for Computing Machinery, Article 53, 6 pages. https://doi.org/10.1145/3649329.3657353

[27] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. Association for Computing Machinery, 65–74. https://doi.org/10.1145/3020078.3021744

[28] Stylianos I. Venieris and Christos-Savvas Bouganis. 2016. fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 40–47. https://doi.org/10.1109/FCCM.2016.22

[29] Deepak Vungarala, Mohammed E. Elbtity, Kartik Pandit, Sumiya Syed, Sakila Alam, Arnob Ghosh, Ramtin Zand, and Shaahin Angizi. 2025. TPU-Gen: LLM-Driven Custom Tensor Processing Unit Generator. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 1–8. https://doi.org/10.1109/ICLAD65226.2025.00010

[30] Bowei Wang, Qi Xiong, Zeqing Xiang, Lei Wang, and Renzhi Chen. 2025. RTL-Squad: Multi-Agent Based Interpretable RTL Design. arXiv:2501.05470 [cs.AR] https://arxiv.org/abs/2501.05470

[31] Olivia Weng, Gabriel Marcano, Vladimir Loncar, Alireza Khodamoradi, Abarajithan G, Nojan Sheybani, Andres Meza, Farinaz Koushanfar, Kristof Denolf, Javier Mauricio Duarte, and Ryan Kastner. 2024. Tailor: Altering Skip Connections for Resource-Efficient Inference. *ACM Transactions on Reconfigurable Technology and Systems* 17, 1, Article 11 (2024). https://doi.org/10.1145/3624990

[32] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. MAGE: A Multi-Agent Engine for Automated RTL Code Generation. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7. https://doi.org/10.1109/DAC63849.2025.11133191

A VERIFICATION, INTEGRATION, AND TOOLCHAIN BUG CATALOGUE

This appendix expands Section 5.3 with the bug taxonomy, the full per-module verification evidence state, and the extended B21 investigation; the complete per-bug index (B1 to B77) is given in Appendix C. Every bug catalogued here was surfaced and resolved by the integration agent (System 2) itself, working from high-level functional directives and its own adversarial multi-agent cross-checks (Section 3.4); the author set objectives and approved milestones but wrote no RTL, ran no commands, and diagnosed none of these faults.

A.1 Bug taxonomy (twelve classes)

We group the 77 indexed bugs into twelve classes by root cause and by the evidence that localises each. Table A.1 also identifies the gate at which each class is exposed.

A.2 Per-module verification evidence state

ResNet-50: all 119 modules register as passing in the pipeline state, and 117 of 119 carry fresh on-disk evidence, of which 108 are bit-identical and 9 are within the three-unit tolerance. The remaining 2 modules lack complete fresh result files because later integration changes altered the testing environment and their original per-module passes were not rerun. Their final per-module evidence is therefore incomplete; nevertheless, the assembled ResNet-50 backbone passes the stricter byte-exact network-level gate.

MobileNetV2: the recorded System-1 cost snapshot covers 97 of the 99 modules, of which 82 are bit-identical and 15 are within tolerance. The global-average-pool and classifier modules were added afterwards, and the complete network passes the assembled network gate (Section 5.3). The per-module bit-identical and within-tolerance split for those two deferred modules is not separately recorded.

A.3 Extended investigation: B21

During end-to-end verification of ResNet-50, a sparse error affecting roughly 2.7% of outputs appeared in wide-channel residual additions in the network's later stages, present only in the composed chain; every implicated module had passed its per-module gate in isolation. Localising it required an eight-hypothesis adversarial code review across 83 handshake-fed nodes. The review first attributed the error to a spatial_run handshake asymmetry and applied a fix that gated valid_in by spatial_run across those nodes, recorded as B21. That attribution was later superseded: the true defect was a missing per-layer activation requantisation in 22 of 48 ReLU modules (B7), where the template had emitted a plain $\max(0, x)$. The B21 handshake change was retained only because it independently reduced cycle count by about 12%. The episode shows that, even after the assembled network gate flags a composed-design failure, root-cause attribution can be hard and a plausible but incorrect fix can mask the true cause, so the byte-exact comparison rather than any single hypothesis remained the arbiter.

B ADDITIONAL SYSTEM DETAILS

B.1 Per-module failure classification and pattern reuse

The per-module pipeline classifies each generation failure into one of sixteen recurring arithmetic and structural error classes (for example integer overflow, missing saturation, or unfolded batch-normalisation), augmented by four infrastructure classes that separate genuine RTL defects from toolchain problems. This failure-memory and pattern-reuse cache retains the broken RTL and the full failure history, so later modules on related layers begin from accumulated experience. When a module passes, it also stores a pattern note with reference RTL; a note is promoted to active guidance after it has helped three later modules and archived if a module that later uses it fails, with promotion gated on the module still passing the final verification and Vivado checks. This is a reuse mechanism rather than a measured self-optimisation.

B.2 System-1 agents and cost

Table B.1 lists the System-1 components and their pinned models; the roles are described in Section 3.2. Four are language model agents (Foundry, Surgeon, Failure-Classifier, Retrospector), and architecture mapping runs as a deterministic ONNX frontend. The model column abbreviates the common `claude-` prefix.

Table B.2 reports the System-1-only pipeline cost discussed in Section 5.1; the MobileNetV2 figures cover the 97 modules in the recorded cost snapshot.

B.3 Derived efficiency

Table B.3 gives the energy and area efficiency derived in Section 5.2. These values require no additional synthesis, and energy figures are comparable only within a device.

B.4 Shared compute engine

The shared time-multiplexed engine, like all of the generated hardware, was produced by the integration agent (System 2) and was not specified by the human: it comprises a 256-lane signed multiply-accumulate array feeding 256 wide accumulators, a three-stage requantisation pipeline, an address generator, a configuration-register block, and a memory-to-stream bridge. Its structure was the agent's own; the author set goals and approved milestones but supplied no wiring diagram or module specification.

B.5 Objective-directed refinement

The improve operation supplies a verified module and its Vivado synthesis report to a language model agent with one of seven predefined PPA objectives (use-DSP, use-BRAM, reduce-LUT, reduce-FF, improve-Fmax, reduce-latency, increase-throughput) and a fixed per-objective guidance prompt. Each candidate is reverified with Verilator and resynthesised, and is accepted only when a deterministic objective-specific threshold is met. Table B.4 lists representative verified runs.

Table A.1. Bug taxonomy over the 77 indexed bugs, with detection gate and localisation evidence. B51 spans two classes, so the counts sum to 78.

#	Bug class	*n*	Detection gate	What localises it
1	Stale-artifact / regeneration omission	7	assembled network	fresh self-consistent golden; artifact mtimes; full regeneration chain
2	Missing or wrong activation-rescale arithmetic	2	assembled network	un-confounded bracket; fit the transfer over the whole input domain
3	Operand half-swap, scaling and tiling layout	8	assembled network	operand probe at the transfer handshake; offline golden replay
4	Handshake / backpressure / deadlock	17	assembled network	live simulation-state inspection; freeze signature (downstream ready, no valid output)
5	Latency and memory-read-latency contract	4	module / assembled	exact-latency check; engine-isolation harness with controllable read latency
6	Simulation-artifact false-positive	4	verification	Verilator `-x-initial 0`; single-threaded run; mapped primitive counts
7	Contract / bus-width / packing	6	assembled network	port width versus contract map; coherent multi-stage patching
8	Engine datapath arithmetic / counter-leak	5	assembled network	single-pixel-isolation testbench; per-pixel grid; drop-multiply-accumulate model
9	Component byte-exact but end-to-end wrong	9	assembled network	per-node probe bisection; Python triangulation
10	Golden-reference float-accumulation artifact	2	verification	compare two scale choices; persistent residual implicates the golden
11	Autonomous-pipeline pattern-reuse and toolchain hazards	12	preflight / PPA	structural gates; infrastructure separation; raw-evidence rubric
12	Frontend / accuracy-measurement	2	measurement	deployment integer path; batch-normalisation identity; scale-ratio check

Table B.1. System-1 components and pinned models.

Component	Input	Job and output	Model
Architecture mapping	Quantised checkpoint	Trace graph, fold batch normalisation, and emit weight/bias hex and the Layer IR via deterministic `read_weights`; outputs the Layer IR once at start-up	deterministic
Foundry	One layer specification	Generate synthesisable signed-integer Verilog with memory-init weights, valid/ready, exact latency, and no simulation-only constructs; outputs metadata and RTL via `write_verilog`	`opus-4-7`
Surgeon	Failing module, result, and history	Classify the fault and rewrite only faulty lines while preserving the interface; outputs repaired metadata and RTL via `write_verilog`	`opus-4-7`
Failure-Classifier	Module failure evidence	Produce the retry-or-abort policy category	`sonnet-4-6`
Retrospector	Failure history, prior RTL, and pattern-reuse knowledge	Inject one advisory into a resumed Foundry session without writing RTL	`opus-4-7`

Table B.2. Full-network System 1 pipeline economics.

Network	Total LLM cost (USD)	Cost per module (USD)	Avg attempts per entry	Modules passed
ResNet-50	170.61	1.43	1.09	119
MobileNetV2	196.39	2.02	1.30	97

Table B.3. Efficiency derived from the routed figures.

Design	Device	fps/W	mJ/inf	fps/kLUT
ResNet-50 (INT3/INT4)	U250	0.92	1087.7	0.012
MobileNetV2 (INT8)	U250	8.43	118.58	0.290
ResNet-8 nn2rtl (INT8)	ZCU104	1,325	0.75	62.7
ResNet-8 FINN (W4A4, max-fold)	ZCU104	3,575	0.28	510.8

Table B.4. Representative verified refinement runs; these isolated demonstrations were not carried into the final routed networks.

Module	Objective	Resource change	Acceptance
node_conv_248	use DSP	1 → 25 DSP	DSP ≥ 8
node_conv_284	reduce LUT	227,703 → 64,530 LUT	≥5% cut
node_conv_290	use BRAM	0 → 7 BRAM	BRAM > 0
node_conv_292	reduce FF	267,737 → 10,357 FF	≥10% cut
node_conv_298	reduce FF	267,918 → 6,254 FF	≥10% cut

C FULL BUG INDEX (B1 TO B77)

The complete catalogue is given in Table C.1.

Table C.1. Complete index of the 77 indexed bugs. ★ marks spine bugs.

ID	Class	Net	Symptom	Fix
B1	1	RN50	14 engine convs systematically low (~16% e2e error), small +bias at final ReLU	rebuild bias_memory_map (stale per-tensor bias.mem); engine real-mem isolation harness exonerated the RTL
B2	1	RN50	e2e numbers fluctuated, byte-exact milestones not reproducible, ~16% error hidden	refresh_final_golden.py retiles a fresh logical golden into the stale contract golden
B3	1	RN50	windowed 3 × 3 convs saturate (max 7f) while 1 × 1 clean	regen_mp_k_weights.py regenerated stale wide tiled packings (read via $readmemh)
B4	1	RN50	saturation explosion (add_9 0→7625), relu_48 saturated	extend the add-rescale patcher to Style-B (FUSED_LHS_MULT) adds it had silently skipped
B5	1	both	engine wrong addresses / range-assert under mixed INT3 packing	reorder build_weight_memory_map, dedup_engine_banks, nibble_int3 [SUPERSEDED]
B6	1	RN50	part of a multi-day ~16% e2e bug (test-only)	single-file repack with –wgt-bits 4 (INT3 wide hex read under WGT_BITS(4) wrappers)
B7 ★	2	RN50	diffuse e2e error, conv_200 93.9% wrong, near-orthogonal final map	add per-layer requant to the 22 of 48 rescaling ReLUs (template emitted a plain $\max(0, x)$)
B8	2	RN50	byte-exact on vec0, mismatching bytes on vec1	re-fit each ReLU transfer over all 128 inputs via a bounding interval (was vector-overfit)
B9	3	RN50	residual error mean|d| ≈ 2.7 onset conv_252 → ~4% at final ReLU	swap the reversed data_in halves of node_add_7 (lone offender among 16 adds)
B10 ★	3	MBV2	all 10 residual adds wrong despite both operands byte-exact and aligned	fix concat order to lhs=low=skip / rhs=high=main in build_top_wrapper (was {skip,main})
B11	3	RN50	tiled adds (OC≥1024) fail ~22%, RTL passes pure-Python sim	add to isTiledAdd plus per-beat lhs|rhs interleave retile (tooling bug, not RTL)
B12	3	MBV2	conv_818 (first DW after dispatch) 100% wrong, stem byte-exact	rewrite g_w_lt to one 2048-bit word per beat, deepen act mem, remap D0/D1 bases
B13	3	MBV2	conv_878 (576-ch) correct to ch319 then wrong from ch320; same on 960-ch	add pull_idx register decoupling resident-beat index from pull advance (off-by-one is_last)
B14	3	RN50	every weight read wrong, engine garbage	Path D: 8 native 288-bit URAM banks (288-vs-2048 width, 8 × 288 ≠ 2048, >>3 skip)
B15	3	RN50	IC>256 layers re-read ch0; 8 of 14 heavy layers wrong dot products	read addr = base + (in_r·IW+in_c)·ic_chunks + ic_chunk_idx (was no IC stride)
B16	3	RN50	every bias byte-swapped to a huge wrong value (−4 → −50,331,649)	struct.pack('>i') big-endian (was '<i', LSByte landed high)

continued on next page

Table C.1. (continued)

ID	Class	Net	Symptom	Fix
B17 ★	4	RN50	chain-wide lockstep freeze, no frame ever emitted	comprehensive backpressure (pulse producers asserted valid, ignored ready)
B18	4	RN50	after backpressuring 25 ReLUs, freeze byte-identical	backpressure must cover every producer (loss relocated to pulse-style conv_200)
B19	4	RN50	subset of pointwise convs sped up → fast/slow imbalance, beat loss	fix param-grep (31 serial 1 × 1 convs existed, only 12 parallelised)
B20 ★	4	RN50	frame never completes; wide 2048-ch ReLUs drop last beat (3135/3136)	drop spatial_run from the ReLU→skid transfer so always-ready skids accept every beat
B21	4	RN50	sparse ~2.7% broad-channel residual, stages 3-4, in-chain only	gate valid_in by spatial_run across 83 skid-fed nodes [SUPERSEDED by B7; kept ~12% cycles]
B22	4	RN50	MP 16→32 on 38 convs hard-deadlocks e2e (output stuck 0)	proposed symmetric output skid on conv_202→add LHS; kept byte-exact MP=16 baseline
B23	4	RN50	deadlock entering final stage (skip FIFOs 0-6 full, 7-15 empty)	revert one-sided tail-pipe (+3 spatial vs +0 engine desynced the add join)
B24	4	MBV2	dispatch-1 deadlock, bridge waits on an inflated tile count	set engine_output_bridge TILES_PER_BEAT=1 (engine writes 1 position per 2048-bit beat)
B25	4	MBV2	dispatch 21 deadlock, loader at full capacity from ~17 real positions	expose wsel_empty as wr_accept, re-gate 19 retile-bridge producers (duplication)
B26	4	MBV2	engine-top parks in S_WAIT_DRAIN, drain_complete never asserts	param-gated output-backpressure primitive (store-and-drop FIFO overflowed, dropped beats)
B27	4	MBV2	engine-top deadlock dispatch 0, scheduler stuck in S_WAIT_LOAD	apply_loader_word_resize.py (capacity in output beats vs 2048-bit words, off by 2048/BUS_W)
B28	4	MBV2	e2e deadlock with backpressure retile bridges	per-bridge spatial_run_drain_br_i mask (a shared global any_retile_stall dropped beats)
B29	4	MBV2	e2e timeout when all 17 DW at MP=16 (also MP=8)	revert conv_884/908 to MP=4 [SUPERSEDED: MP16 later byte-exact once bridges deleted]
B30	4	RN50	engine FSM enters ST_REQUANT, never advances, deadlock	remove 8 datapath tie-offs outside the ifndef guard, wire from addr-gen and bias mem
B31	4	RN50	AXI4-Lite write never completes, engine never configured	collapse S_WRITE_ADDR and S_WRITE_DATA into one S_WRITE asserting awvalid+wvalid the same cycle
B32	4	RN50	engine never starts, design does nothing on input	add scheduler instance, wire AXI master→slave, drive engine_start (was tied 0)
B33	4	MBV2	stem freeze / e2e never completes, ~50% beats dropped	elastic-pipeline rollout across all spatial producers (was push-only, no backpressure)
B34 ★	5	RN50	small ±1.. ± 12 in-chain errors on engine convs, isolation sims passed	align engine to 2-cycle weight read (URAM READ_LATENCY_A=2 vs 1-cycle pipeline = stale weight)
B35	5	MBV2	engine output catastrophically wrong, 2048-bit bus truncated to 1024	override engine params WGT_W=8 / URAM_DATA_W=2048 / WLAT=2 (defaulted to ResNet INT4)
B36	5	RN50	clean mismatch with timing_pass=true, ~0.5-1% sign-skewed	change the prefetch guard oc_pass+2 < OC_PASSES to <= (stale double-buffered cache)
B37 ★	6	RN50	deep conv reads ~94-96% wrong vs golden in many harnesses	run Verilator −x-initial 0 (uninitialised FF X-poisoning; true residual only 2.72%)

continued on next page

Table C.1. (continued)

ID	Class	Net	Symptom	Fix
B38	6	MBV2	~688 wrong logit bytes under –threads 4, cycle count unchanged	pin the runner to –threads 1 (multithread scheduler cross-partition hazard)
B39	6	RN50	long-standing ±1 error on spatial conv_216	drive the undriven window_kwm1_wire in line_buf_window.v (lint-flagged UNDRIVEN)
B40	6	RN50	functional sim reads correct URAM, falsely implying init works	inspect mapped primitive count (synth fell back to RAMB36E2; sim exercised BRAM not URAM)
B41 ★	7	MBV2	both tops consume all input, produce no output (final-stage deadlock)	contract-regen major phase: retile bridges (576/960/1280-ch > 4096-bit flat-bus cap)
B42	7	MBV2	correcting one high-OC stage deadlocks the next; chain regresses	coherent multi-stage contract patching [SUPERSEDED: region later byte-exact]
B43	7	RN50	every layer's requant output garbage (scale high bits discarded)	widen SCALE_MULT_W 16→32 (ResNet scales ~30 bits, e.g. 1284434803)
B44	7	RN50	±1 errors on rounding boundaries accumulating through depth	apply_scale_constant_fix.py re-derive 21 convs' mult/shift [SUPERSEDED for engine ROM; spatial kept]
B45	7	RN50	engine FSM never leaves requant for layers narrower than max OC	change ST_REQUANT exit to oc_pass_idx == oc_pass_total_m1[2:0] (was hardcoded ==7)
B46	7	RN50	SELRANGE warnings (×10) on 1 × 1 / non-MP_K==9 conv instances	gate the channel_select assign in generate (USE_CHAN_WINDOW=1 else tie 0)
B47 ★	8	RN50	layer 99.9960% byte-exact, 2/50,176 wrong at pixel [1,4] (off by −1)	re-gate weight/act read on ~mac_done not ~k_at_last (suppressed the last MAC tuple)
B48 ★	8	RN50	OCs off by the ic=0 contribution, mismatch 2..8578 on 9 of 14 layers	wrap counter-advance in if(!mac_done) (ic_cnt leaked 0→1, dropped first MAC)
B49 ★	8	RN50	same bug = 0 mismatches on some layers, thousands on others	(B48 fix): visibility is data-dependent, mismatch tracks the count of a[ic=0]≠0 pixels
B50	8	RN50	engine convs off-by-one (got=gold−1) on negative half-boundaries	unconditional +HALF round-half-up (was sign-aware HALF / HALF-1)
B51	5/8	RN50	engine dropped the last MAC of every output-channel pass	re-gate counter-advance by ~mac_done not ~k_at_last (one bug, two class lenses)
B52 ★	9	both	sub-blocks each pass local gates but do not work wired together	3 audit rounds fixed 12 cross-piece bugs (AXI / scale-trunc / URAM-bus / bias / FSM-exit / byte-order)
B53 ★	9	MBV2	modules byte-exact in isolation, integrated chain corrupts from 1st dispatch	march a per-node e2e probe (iso goldens were consistent-with-the-bug); e2e mismatch is the primary functional oracle in simulation
B54	9	RN50	byte-correct per-OC conv still reports mismatch via equiv_one	DBG_SCALE dump + Python triangulation localised to the tiling path; mandate in-chain e2e
B55	9	RN50	all conv outputs zero (got=0) in sim	make SCALE_PATH absolute (relative path did not resolve in sim cwd → scale ROM zero)
B56	9	RN50	hard all-zero from the first engine-fed residual add onward	run the executable with cwd at repo root ($readmemh relative paths not found → mems zero)
B57	9	RN50	known-good conv_198 equiv_one max_error 75, design ok in-chain	triangulate_conv198.py recompute matched the logical goldout → fault is the RTL path only
B58	9	RN50	an earlier review concluded the deployed weights were INT8	adversarial re-scan: the _wide INT8 files are dead (engine computes from the INT4 URAM bank)
B59	9	MBV2	conv_820 wrong despite byte-exact spatial input, ~99% downstream	set scheduler act_out_base[D1]=12544 (a dead act_out write collided with ldr2/ldr3 windows)

continued on next page

Table C.1. (continued)

ID	Class	Net	Symptom	Fix
B60	1/9	RN50	conv_248 stage-3 residual off ±1-6 (equiv_one max_error up to 51)	verify known-good stage-1/2 conv vs fresh INT4 goldens; deferred pending consistent regen
B61	9	MBV2	per-module off-by-one (max_error=1) on 17 of ~17 flagged	canonical compute_scale_approx + unconditional bias + shift bumps
B62	10	MBV2	node_linear 2/8000 off-by-one logits	harden golden_impl Int8Gemm to integer requantize (golden cast acc to float32; the RTL is correct)
B63	10	MBV2	a generic integer-hardened golden emits bytes the RTL will not (119 vs 118)	harden only node_mean (7619,18); flag and skip relu/add (per-module agent-chosen constants)
B64	11	both	a working contract permanently flagged manual_correction_needed	add a toolchain_infra class + retryable fallbacks (transient infra is not an RTL failure)
B65	11	both	contract switched but verified against flat-bus goldens (wrong protocol)	executable plans + per-contract golden retile/repack + latency [SUPERSEDED: now executable]
B66	11	both	a later degraded repair becomes the cached state, the better design lost	gate cache promotion on the module still passing after the Vivado/PPA gate
B67	11	MBV2	grouped/depthwise/dilated conv verifies against wrong math	add groups/dilation fields to LayerIR + depthwise detection + asymmetric-pad rejection
B68	11	both	the pattern-reuse cache cannot tell which learned docs a module used	deterministic pattern injection + hard-fail/retry when a required lookup is absent
B69	11	both	a valid MaxPool omitted by stale extraction instructions	route extraction through deterministic read_weights [SUPERSEDED]
B70	11	both	a file looks stale/missing depending on Win/WSL/native opener	normalise all path forms at tool boundaries + content-hash fingerprint identity
B71	11	RN50	pipeline repeatedly repairs correct RTL (phantom failures, $7.50)	harden run_iverilog to emit a structured no-diagnostic; classify as tb_setup_error/toolchain_infra
B72 ★	11	both	Verilator passes but synth preflight fails on an async-reset array write	split the memory write into a dedicated always @(posedge clk)-only block (restore RAM inference)
B73	11	both	Verilator never terminates, hits the wall-clock cap	add VERILATOR_SIM_TIMEOUT_MS + an output_counter_missing structural preflight rule
B74	11	RN50	stem first_mismatch ≈7122, max_error 1-3, errors in the last quarter	move the wrap math (IW-1+PW) into coord_scheduler.v + right-pad from zero BRAM cells
B75	12	RN50	measured top-1 ~0% or a misleading ~73% → false "design broken"	measure via the onnx_frontend int path / BN-identity injection (ONNX is BN-folded, double-counts)
B76	12	both	Vitis HLS launches, idles indefinitely at ~0% CPU, no csynth	plain float ONNX + per-layer ap_fixed (move_scales explodes scalar → per-element constants)
B77	11	both	repair agent steered to add drain logic, real bug = off-by-one exit	replace the TB prose hypothesis with raw mismatch numbers + a general interpretation rubric

Note: ★ marks spine bugs. B51 (class 5/8) is the only bug counted under two classes, which is why the per-class counts in Table A.1 total 78 for 77 distinct bugs; B60 is labelled 1/9 but counted once, under class 1.

D PIPELINE AND VERIFICATION DIAGRAMS

Figure D.1 details the System-1 per-module pipeline state machine and its failure-memory and pattern-reuse loop (Section 3.2); Figure D.2 details the two functional-verification gates and the reference-independence principle (Section 3.3).

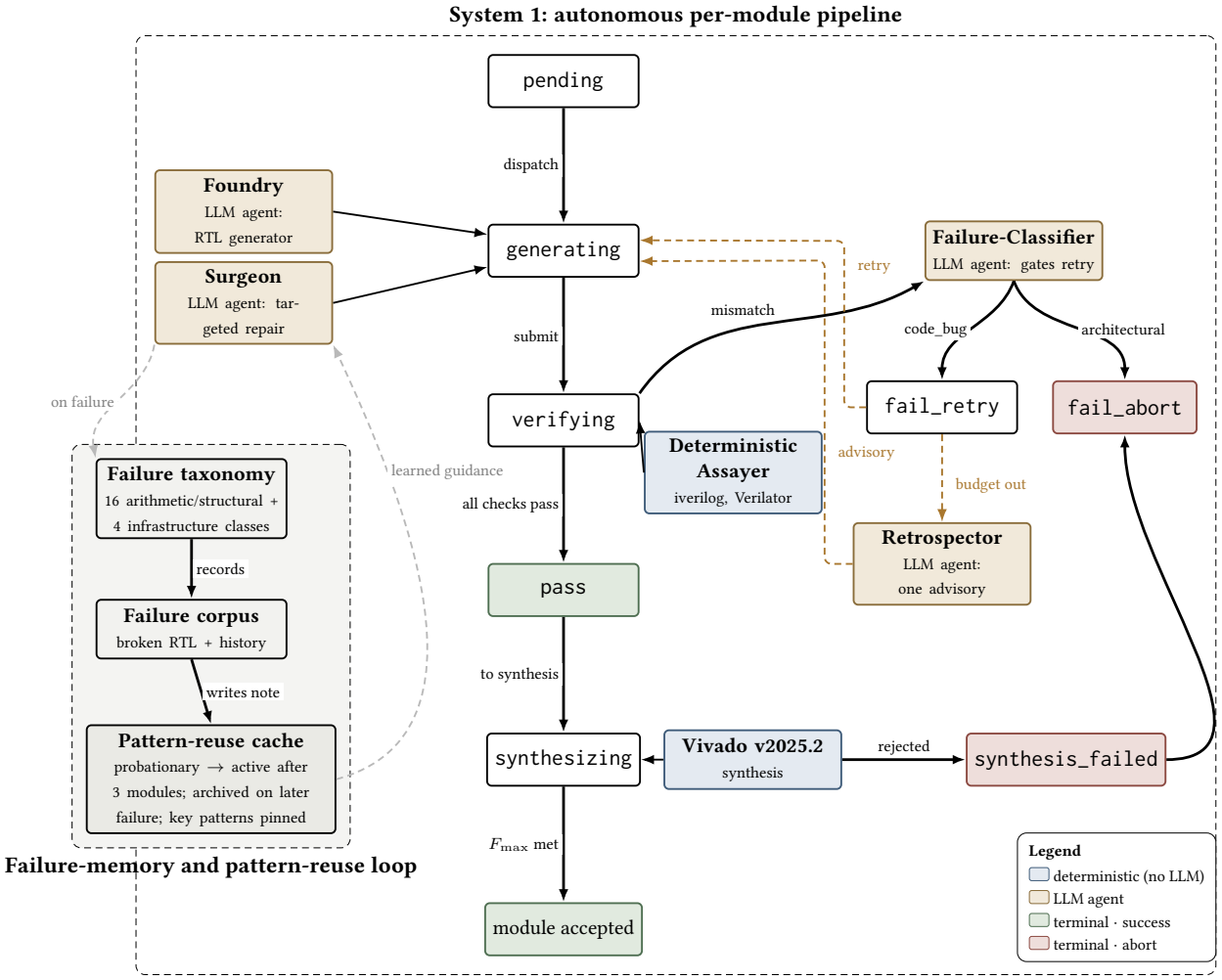

Fig. D.1. The System-1 per-module pipeline as a state machine, with Vivado synthesis dispatch and the failure-memory and pattern-reuse loop. Deterministic components are shaded separately from the LLM agents.

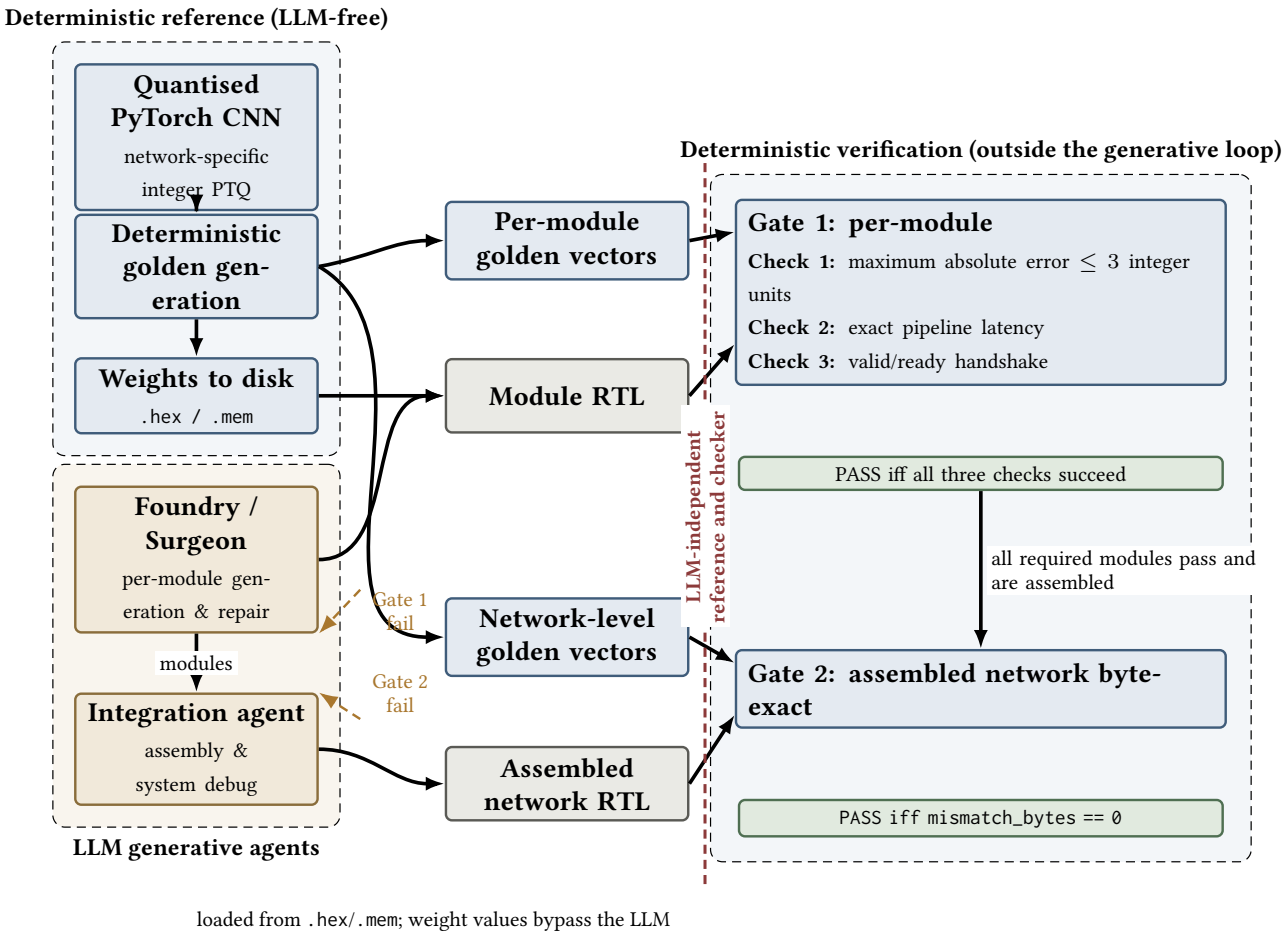

Fig. D.2. The two-gate verification methodology: a per-module Gate 1 and a stricter byte-exact Gate 2 over the assembled network. The reference and checker are produced without an LLM, preserving the "two-bug" independence principle.